Lab 8: Introducing Hyperparameter Tuning

Objectives:

- To gain hands-on experience tuning parameters
- To implement concepts related to Hyperparameter Tuning

Lab

Load Necessary Libraries

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Load and Prepare Data

We're going to use iris dataset.

In [2]:
```
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split into training and testing sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Let's use decision tree again! (Don't forget it yet)

In [3]:
```
# Initialize a Decision Tree model
dt = DecisionTreeClassifier(random_state=42)
```

Parameter Tuning Using GridSearchCV

In [4]:
```
# Define parameter grid for tuning
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Apply GridSearchCV
grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get best parameters
print("Best Parameters from GridSearchCV:", grid_search.best_params_)
```

```
Best Parameters from GridSearchCV: {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_spli
t': 2}
```

Parameter Tuning Using RandomizedSearchCV

In [5]:
```
from scipy.stats import randint

# Define parameter distributions for tuning
param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': randint(2, 20),
```

```
    'min_samples_leaf': randint(1, 10)
}

# Apply RandomizedSearchCV
random_search = RandomizedSearchCV(dt, param_dist, n_iter=10, cv=5, scoring='accuracy', n_jobs=-1, random_state=42)
random_search.fit(X_train, y_train)

# Get best parameters
print("Best Parameters from RandomizedSearchCV:", random_search.best_params_)
```

```
Best Parameters from RandomizedSearchCV: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 3, 'min_samp
les_split': 3}
```

Evaluate the Best Model

In [6]:
```
# Train a Decision Tree using the best parameters from GridSearchCV
best_dt = grid_search.best_estimator_
y_pred = best_dt.predict(X_test)

# Compute accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy with Best GridSearchCV Model: {accuracy:.4f}")
```

```
Test Accuracy with Best GridSearchCV Model: 1.0000
```

Tasks:

Use wine dataset :)

In [7]:
```
from sklearn.datasets import load_wine

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target
```

Task 1: Train-Test Split

- Split the Wine dataset into training and testing sets using an 80-20 split.
- What is the role of the train-test split in evaluating a machine learning model?

In [8]:
```
# Write your code here
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Ans: Training set is use for learning about Pattern of Data set

Testing set is use for evaluating a machine learning after training and it is unseen data

Task 2: Cross-Validation

- Implement cross-validation using GridSearchCV and RandomizedSearchCV.
- How does cross-validation help in providing a more reliable estimate of the model's performance?
- Discuss how cross-validation improves the results and prevents overfitting compared to a simple train-test split.

In [9]:
```
dt = DecisionTreeClassifier(random_state=42)
```

In [10]:
```
# Write your code here
# GridSearchCV
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(dt,param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)
```

```
print("Best Parameters from GridSearchCV:", grid_search.best_params_)
print(f"Best Accuracy: {grid_search.best_score_:.4f}")
```

```
Best Parameters from GridSearchCV: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split':
2}
Best Accuracy: 0.9224
```

In [11]:
```
# RandomizedSearchCV
param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10)
}

random_search = RandomizedSearchCV(dt, param_dist, n_iter=10, cv=5, scoring='accuracy', n_jobs=-1, random_state=42)
random_search.fit(X_train, y_train)

print("Best Parameters from RandomizedSearchCV:", random_search.best_params_)
print(f"Best Accuracy: {random_search.best_score_:.4f}")
```

```
Best Parameters from RandomizedSearchCV: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 5, 'min_samp
les_split': 2}
Best Accuracy: 0.9150
```

Ans: Cross-validation (CV)

- ช่วยทำให้เกิดความน่าเชื่อถือ เนื่องจากเป็นการแบ่งข้อมูลออกเป็นหลายส่วนใช้ในการ train หลาย ๆ รอบ และนำ accuracy ของแต่ละรอบมาคิดเป็นค่าเฉลี่ย
- ช่วยป้องกัน Overfitting ถ้าเกิดการ overfitting คะแนนที่ได้ในแต่ละรอบจะไม่คงที่เนื่องจากทำคะแนนได้ดีแค่ในบางรอบ แต่ถ้าใน Simple train-split เราจะไม่สามารถรับรู้ได้

Task 3: Hyperparameter Tuning

- Use GridSearchCV to find the best hyperparameters by exhaustively searching through the parameter grid.
- Use RandomizedSearchCV to sample a fixed number of combinations of hyperparameters.
- Compare the results from both methods in terms of accuracy and computation time.
- Discuss the trade-offs between GridSearchCV and RandomizedSearchCV.

In [12]:
```
# Write your code
import time

param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

start = time.time()
grid_search = GridSearchCV(dt, param_grid, scoring='accuracy', cv=5, n_jobs=-1)
grid_search.fit(X_train, y_train)
grid_time = time.time() - start
```

In [13]:
```
param_dist = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10)
}


start = time.time()
random_search = RandomizedSearchCV(dt, param_dist,
                                   n_iter=20, cv=5, n_jobs=-1, random_state=42)
random_search.fit(X_train, y_train)
random_time = time.time() - start
```

In [14]:
```
print(f"GridSearchCV:")
print(f"  - Best Accuracy: {grid_search.best_score_:.4f}")
print(f"  - Computation Time: {grid_time:.4f} seconds")
print(f"  - Best Params: {grid_search.best_params_}")
```

```
print(f"\nRandomizedSearchCV:")
print(f"  - Best Accuracy: {random_search.best_score_:.4f}")
print(f"  - Computation Time: {random_time:.4f} seconds")
print(f"  - Best Params: {random_search.best_params_}")

time_diff = grid_time / random_time
print(f"\nGrid Search ใช้เวลานานกว่า Random Search ประมาณ {time_diff:.2f} เท่า")
```

```
GridSearchCV:
  - Best Accuracy: 0.9224
  - Computation Time: 0.3879 seconds
  - Best Params: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}

RandomizedSearchCV:
  - Best Accuracy: 0.9155
  - Computation Time: 0.1197 seconds
  - Best Params: {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 3, 'min_samples_split': 15}

Grid Search ใช้เวลานานกว่า Random Search ประมาณ 3.24 เท่า
```

Ans Grid Search ได้ accuracy 0.9224 แต่ใช้เวลามากกว่า

- ใช้เวลาและทรัพยากรมากกว่า การันตีที่จะได้ parameter ที่ดีที่สุดในช่วงที่กำหนดไว้

Randomized Search ได้ accuracy 0.9155 ใช้เวลาน้อยกว่า

- ใช้เวลาและทรัพยาการน้อยกว่า แต่จะเป็นการสุ่ม parameter ไม่ได้การันตีว่าจะได้ผลลัพธ์ที่ดีที่สุด

Task 4: Model Evaluation

- Using the best model from GridSearchCV or RandomizedSearchCV, make predictions on the test set and calculate the accuracy.
- Compare the performance of the tuned model with a baseline Decision Tree model (without hyperparameter tuning).
- How does hyperparameter tuning affect the accuracy of the Decision Tree model?

In [15]:
```
# Write your code here
# best model of GridSearchCV
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)

# Baseline Model
baseline_model = DecisionTreeClassifier(random_state=42)
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)
accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
```

In [19]:
```
# เช็คความลึกของต้นไม้ (Tree Depth)
print(f"Baseline Tree Depth: {baseline_model.get_depth()}")
print(f"Tuned Tree Depth: {best_model.get_depth()}")
# เปรียบเทียบ Training vs Test Score
print(f"Baseline: Train {baseline_model.score(X_train, y_train):.4f} | Test {accuracy_baseline:.4f}")
print(f"Tuned   : Train {best_model.score(X_train, y_train):.4f} | Test {accuracy_tuned:.4f}")
```

```
Baseline Tree Depth: 4
Tuned Tree Depth: 3
Baseline: Train 1.0000 | Test 0.9444
Tuned   : Train 0.9930 | Test 0.9444
```

ถ้าดูจาก accuracy อาจจะคิดว่าาต้องอันนี้เหมือนกัน แต่ถ้าบอกลึกลงไปที่ Train accuracy จะเห็นว่าของ baseline จะได้ 1.00 หรือดูจากความลึกของต้นไม้ที่ลึกถึง 4 level แต่ของ Tuned model จะเห็นว่าลึกแค่ 3 level และได้ accuray เพียง 0.993 ซึ่งหมายความว่าตัว baseline model ได้ทำการสร้างต้นไม้ที่เกินการ overfit แต่ของ Tuned model แม้ Train accuracy จะได้คะแนนน้อยกว่าแต่ใน Test กับได้คะแนนเท่ากัน แสดงให้เห็นว่ามีความ Generalization มากกว่า